Language-Guided Legged Navigation: Technical Design and Build Plan

Sep 30, 2026 · @Damien Bardina

Mission

A simulated Unitree Go1 receives an instruction such as "go to the platform next to the blue cone" and must reach that platform across a stepping-stone field where the straight route is sometimes impossible.

Three layers in a chain do the work. A language model picks the target (the ears), A* picks a route through the stones and cuts it into short hops (the map-reader), and a learned policy walks each hop and chooses its own footholds (the legs).

Pitch: Zhang et al. built the legs; we add the ears and the map-reader, and measure what breaks when they are connected.

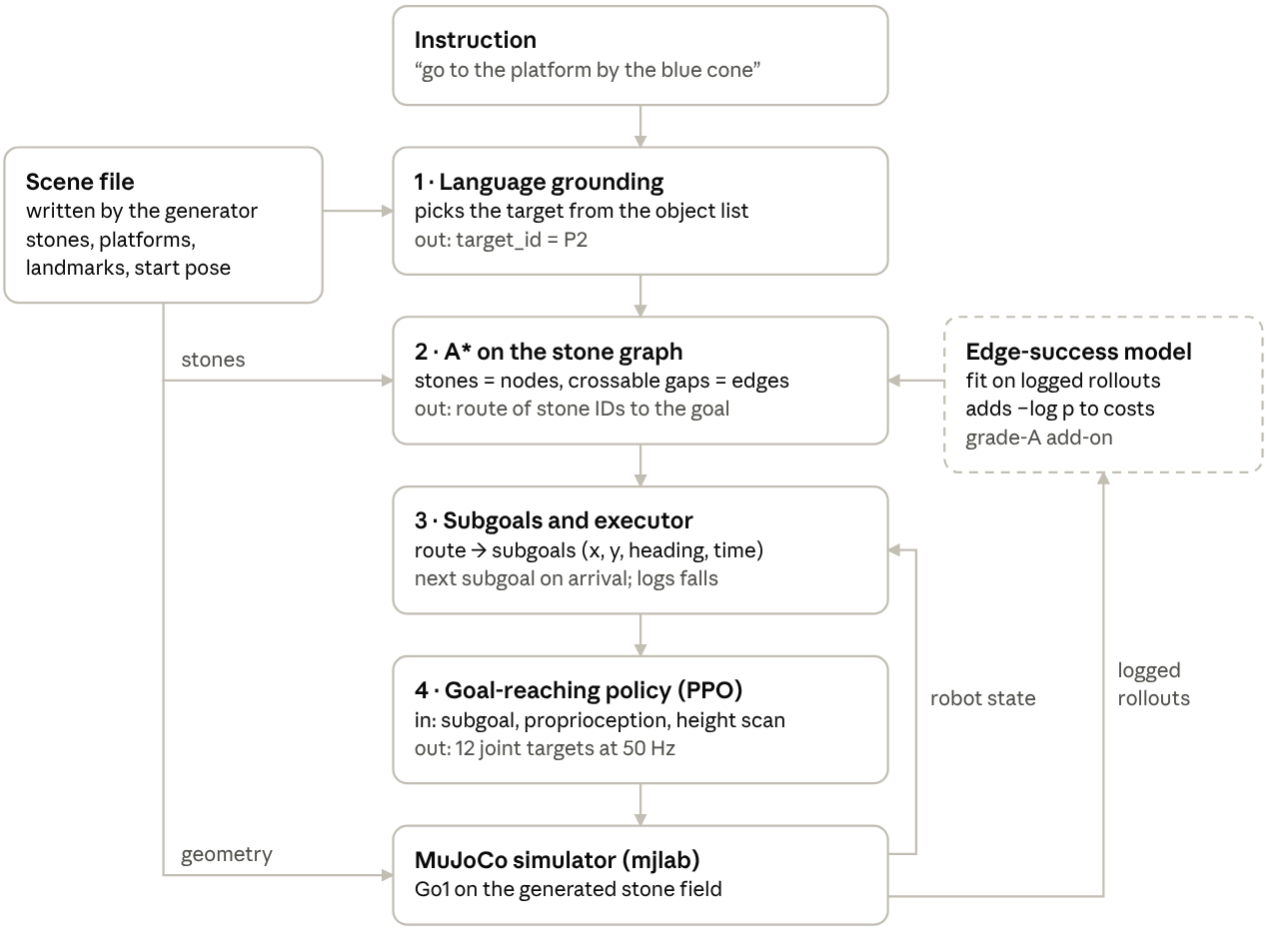
What it delivers: a scaled-down, simulation-only replication of their goal-reaching controller, plus four things they did not study. These are fields that force detours beyond the robot's view, goals that come from language, a planner whose costs reflect the controller's measured success, and per-layer failure attribution.

Scope: a systems study at student scale, not a new locomotion method.

System overview

Four stages run in a fixed order in every episode. Only the controller is trained with reinforcement learning; the others are a language model, a classical search and a small state machine.

Language picks the goal, A* the route, the policy the footholds



system overview · four stages, one scene file, one add-on

The scene file feeds every layer. The dashed edge-success model is off in the baseline system and switched on only for the execution-aware experiment.

Interface contract, fixed on Day 1. Every arrow above is one of these messages, and changing any of them after Day 1 needs all three people to agree.

From, to	Message	Format
Generator, all layers	scene file	JSON: stones, platforms, landmarks, start pose, seed, family, level
Language, planner	target platform	`{"target_id": "P2"}` or `NONE`
Planner, executor	route and subgoals	list of stone IDs; list of `{x, y, heading, time_budget}`
Executor, policy	command at each control step	target (x, y) in the robot's frame, heading error, time left

From, to	Message	Format
Executor, evaluation log	one row per episode	outcome (success, fall, timeout), failing subgoal index, failing layer

Scene and terrain

Every episode runs on a generated stone field, and the generator writes the same layout to a scene file. That file is the single source of truth: the language layer reads its objects, the planner reads its stones, and the simulator is built from it.

Stone levels. Stones follow Zhang et al.'s parameterisation: stone size, gap, lateral shift and height. The values below start at roughly half of their ANYmal values and are starting guesses; calibrate the hardest level on Day 5 from what the policy manages. Shift and height stay at zero in the core and are reserved as unseen parameters for the generalisation study.

Level	Stone size (cm)	Gap (cm)	Lateral shift (cm)	Height variation (cm)
0	46	4	0	0
5	26	9	0	0
9	10	13	0	0
Unseen test set	10 to 26	9 to 13	up to 6	up to 5

Intermediate levels interpolate linearly. Gaps are pits deeper than the Go1's legs, so a foot in a gap means a fall.

Layout families.

- **Open field** (training, RQ1): a full grid of stones, as in Zhang et al., so walking straight at any goal is always feasible. During training, 0 to 20% of stones are removed at random so that missing stones are familiar.
- **Moat field** (RQ2, RQ3): a band two stones wide is removed across the field, except for one bridge placed at least 2 m to the side of the start-to-goal line. From the direct approach, the bridge is outside the height scan.
- **Dead-end field** (RQ2, RQ3): the direct line leads into a pocket of stones that ends in a gap wider than any crossable edge, so the real route goes around it.

Platforms and landmarks. Each field has one start platform, 3 to 5 goal platforms (larger stones) and 3 to 6 landmarks such as coloured cones and boxes. Landmarks are visual-only geoms with no collision, placed beside platforms. They matter for language and for

videos, not for walking.

Scene file. One JSON file per layout seed:

```
{
  "seed": 1042, "family": "moat", "level": 5,
  "stones":    [{"id": "S17", "x": 2.10, "y": -0.35, "z_top": 0.00, "size":
0.26}],
  "platforms": [{"id": "P2", "x": 6.40, "y": 1.20, "z_top": 0.00, "size":
0.60}],
  "landmarks": [{"id": "C1", "type": "cone", "colour": "blue", "x": 6.80,
"y": 1.30}],
  "start": {"platform": "P0", "heading": 0.0}
}
```

Splits. Layout seeds are divided into train, validation (for tuning subgoal spacing and λ) and test (for reported numbers). No test seed is ever used for tuning.

Layer 1: Language grounding

The model works out on its own which platform the instruction means, but it chooses only among objects listed in the scene file, and it never outputs coordinates. Code looks up the chosen platform's position afterwards.

Why IDs, not coordinates. Small language models are unreliable at arithmetic over raw numbers and can invent plausible-looking positions. Restricting the output to a valid ID makes every error discrete, easy to score and impossible to crash the planner with.

Input to the model. The instruction, plus a text version of the scene: every platform and landmark with its ID, type and colour, and relations precomputed in code. For example: *P2: platform; nearest landmark C1 (blue cone, 0.4 m east); 5.1 m from start; left of P3.* Turning geometry into words in code keeps the model doing language, not geometry.

Output. `{"target_id": "P2"}`, where `target_id` is forced by constrained decoding to be one of the platform IDs or `NONE` (for instructions that name something absent). Use a grammar or JSON-schema decoder such as Outlines, XGrammar, llama.cpp grammars or vLLM structured outputs.

Model. An open instruction-tuned model that fits a Colab T4 (16 GB): a 3 to 4 B model in fp16, or a 7 to 8 B model in 4-bit. Grounding runs once per (scene, instruction) pair and the result is cached, so the language model never has to run inside the simulation loop on the H100.

Perception is out of scope. Object labels and positions come from the simulator, not from a camera. State this plainly in the report as an assumption.

Instruction test set. About 100 to 150 instructions over about 20 scenes, each labelled with the correct platform ID by code:

Category	Example	Share
Landmark relation	"go to the platform next to the blue cone"	40%
Spatial or ordinal	"the farthest platform on the left"	25%
Attribute plus distractor	"the platform by the cone that isn't red"	20%
Unresolvable	"go to the green box" when no green box exists (answer: NONE)	15%

Hold out a set of paraphrases and one relation type (for example "between") for the generalisation study.

Metric and baseline. Grounding accuracy = share of correct IDs, per category. Compare against a keyword baseline that matches colour and type words and picks the nearest platform. The baseline shows what the language model actually adds.

Layer 2: Planner

A* outputs an ordered list of stone IDs from the start platform to the goal platform. A post-processing step turns that list into a handful of subgoals, and the subgoals are what the controller actually receives.

Stone graph. Nodes are the stones and platforms in the scene file. An edge joins two stones when the edge-to-edge gap is at most g_max and the height difference at most h_max. Set g_max from the controller: the largest gap it crosses reliably at the end of training.

Search. Plain, fixed A*. The edge cost is the distance between stone centres, and the heuristic is the straight-line distance to the goal. That heuristic never overestimates, because every edge costs at least its own straight-line length, so A* returns the shortest route. The algorithm and heuristic stay the same in every experiment; only the edge costs change in the execution-aware variant (next-but-one section).

```
c(i,j) = \lVert p_i - p_j \rVert, \qquad h(n) = \lVert p_n - p_{\text{goal}} \rVert
```

Output 1, the route. For example `[P0, S12, S13, S21, S29, S30, P2]`. This is a corridor, not a foothold plan. The policy chooses its own footholds from the height scan; the route only guarantees that a crossable chain of stones exists along it.

Output 2, the subgoals. Walk along the polyline of stone centres and:

1. Place a subgoal on the first route stone at or beyond every d metres of path length (default 1 m, swept in RQ2).
2. Add a subgoal wherever the straight line between two consecutive subgoals would cross a region with no stones. The policy heads roughly straight at its target, so this check stops it being lured across a moat.
3. Give each subgoal a heading (towards the next subgoal) and a time budget (segment length divided by the policy's typical speed, plus a margin).
4. End with the goal platform itself as the final subgoal.

A typical 6 m route gives 5 to 8 subgoals, each `{x, y, heading, time_budget}` and always centred on a stone.

Executor. A small state machine that runs at the policy's control rate:

1. Feed subgoal *i* to the policy as target position in the robot's frame, heading error and time left.
2. When the robot is within the reach radius of subgoal *i* (about 0.3 m), switch to *i + 1*.
3. Success: the robot stands on the goal platform, within its radius, for 1 s.
4. Fall: any termination condition (body contact, base too low, foot in a pit). Timeout: the subgoal's time budget runs out without a fall. Both are logged with the index of the failing subgoal.

The core has no replanning, which keeps failure attribution clean. Replanning from the current stone after a timeout is a stretch goal.

Layer 3: Controller

The controller is a goal-reaching PPO policy for the Go1, set up as in Zhang et al. and trained on open stone fields with chained goals so that it can follow the planner's subgoals. Everything downstream depends on it, so it gets two people from Day 1.

Starting point in mjlab. mjlab ships a Go1 asset and demonstrated a Go1 velocity policy trained on its stair-terrain curriculum (v1.3.0 release). Its RayCastSensor with a grid pattern produces height maps (sensor docs). Fork that task, replace the command and reward terms, and add the stone terrain. On Day 1, check whether a stepping-stone terrain type ships with the pinned version; if not, write a custom generator that emits both the geoms and the scene file. Use mjlab's zero-action and random-action agents to sanity-check the environment before any training (README).

Task formulation. Each episode gives a target position and heading. The main reward is paid only in the last few seconds of the time budget, so the policy is free to choose its own speed and gait on the way.

Observations (actor).

- Proprioception: base angular velocity, projected gravity, joint positions and velocities relative to the default pose, previous action.
- Command: target position in the robot's frame (x, y), heading error as sin and cos, time left.
- Height scan: a ray-cast grid around the base, heights relative to the base. Zhang et al. used 25 × 16 points at 7 cm for ANYmal; start with about 1.2 × 0.8 m at 5 cm for the Go1.
- Base linear velocity goes to the critic. Giving it to the actor is acceptable in a simulation-only project, but state it.

Actions. Joint position targets for the 12 joints (default pose plus scaled action), tracked by PD control, with the policy running at 50 Hz.

Rewards. Start from Zhang et al.'s reward table and rescale for the Go1:

Term	What it does	When active
Position tracking	$1 / (1 + \text{distance}^2)$ to the target	last seconds of the budget
Heading tracking	same form, for heading	when near the target
Termination penalty	large negative on a fall	at the fall
Don't wait	penalises standing still far from the target	always
Move in direction	rewards velocity towards the target	first ~150 iterations only
Stand still	penalises motion once at the target	at the target
Regularisers	joint velocity, action rate, torque, base and foot acceleration, thigh and shank collisions	always
Edge term (optional)	penalises foot contacts within ~2 cm of a stone edge	only if edge slips dominate the falls

Terminations. Contact of base or thighs with anything, base height below the stone tops minus a threshold, a foot touching a pit floor, or the time budget running out.

Curriculum. Use mjlab's terrain curriculum with rows as the levels from the scene section. Promote a robot to a harder level when it reaches its target. Demote it only when it makes almost no progress, which is Zhang et al.'s relaxed rule for sparse rewards.

Chained goals (the interface fix). In about half the episodes, when the robot reaches its target, a new target is sampled 0.8 to 2 m further on and the time budget resets. This makes the executor's subgoal switching part of the training distribution. Without it, the policy only ever learns to walk to one target and stop.

Symmetry augmentation. Mirror every sample left to right and front to back. Zhang et al. found this the most important component for robustness. rsl_rl has a symmetry-augmentation option in recent versions; confirm it in the version mjlab pins and write the Go1 mirroring function for observations and actions.

Left out on purpose. Curiosity rewards (small effect in Zhang et al.) and generalist-to-specialist fine-tuning (there are no specialist terrains in scope).

Budget. Zhang et al. trained their generalist for 8000 iterations with 4096 robots. *Open question:* how many iterations per hour one H100 partition gives in mjlab. Measure it on Day 2, because it decides how many seeds and conditions fit. Run training in resumable chunks of at most 2 h.

Execution-aware edge costs (grade A, Damien)

Nothing inside A* learns, and PPO never touches the search. The algorithm and heuristic stay fixed; what changes is the number on each edge, estimated once from recorded attempts of the frozen policy.

Idea. Plain A* treats a 12 cm gap and a 4 cm gap of equal length as equally good. The policy does not. We measure how often the policy actually gets across each kind of edge and make risky edges more expensive, so A* prefers routes the controller can execute. This is SayCan's principle applied to a route planner: ground the plan in what the robot can do.

Procedure.

1. Freeze a policy checkpoint (the M2 checkpoint first, the final one later).
2. Run plain-A* routes on many training-distribution layouts across all levels, in parallel on the H100, logging the base trajectory and the outcome.
3. Label edges. Map the base position over time onto the route. Edges the robot fully passes are successes; the edge it is on when it falls is a failure; edges never reached get no label.
4. Build features per edge: centre distance, edge-to-edge gap, height difference, both stone sizes, turn angle from the previous edge, local stone density.

5. Fit a classifier for the probability of success: logistic regression first, a small MLP or gradient-boosted trees only if that underfits. Oversample hard layouts, since failures are rare, and check calibration with a reliability plot.
6. Replace the edge cost with the one below and rerun the same A*.

```
c'(i,j) = \lVert p_i - p_j \rVert + \lambda \, \bigl(-\log \hat{p}_{ij}\bigr)
```

Summing −log p̂ along a route gives minus the log of the route's estimated success probability, if edges fail independently. A* therefore trades distance against risk, with λ setting the exchange rate; tune it on validation layouts (for example 0, 0.5, 1, 2). The straight-line heuristic stays valid, because the added term is never negative.

Evaluation. Same policy, same test layouts (moat and dead-end fields), plain versus learned costs: end-to-end success, fall rate and route length. Report calibration of p̂ against the observed edge success as well.

Limits to state. Edge labels are approximate, because the policy picks its own footholds and may not step on the route's stones. The independence assumption ignores that a stumble on one edge makes the next edge harder. On Day 13, collect a fresh batch of rollouts with the final policy and refit the classifier; the fit itself takes minutes.

Evaluation

Every experiment runs through one harness that reads scene files, runs episodes and writes one log row per episode. Each failure is labelled with the first layer that caused it.

Metrics.

Metric	Definition
Success rate	robot stands on the correct goal platform within its radius and time budget, no fall
Fall rate	episode ended by a termination condition
Timeout rate	no fall, but a subgoal's time budget ran out
Time to goal	seconds, successful episodes only
Grounding accuracy	chosen platform ID equals the labelled one
Edge-contact rate	share of foot contacts within 2 cm of a stone edge (replaces foothold error)
Route length ratio	route length divided by straight-line distance

Failure attribution by oracle substitution. Run every test episode twice: once with the language model, once with the correct platform ID given directly. The difference in success is the cost of language errors. For the remaining failures, label in this order:

1. **Language:** the chosen ID is wrong (or NONE when a target exists).
2. **Planner:** no route found although the graph has one, a route edge above g_max, or a timeout on a segment whose straight line crosses a region with no stones.
3. **Controller:** a fall or timeout on a valid segment.

Research questions and experiments.

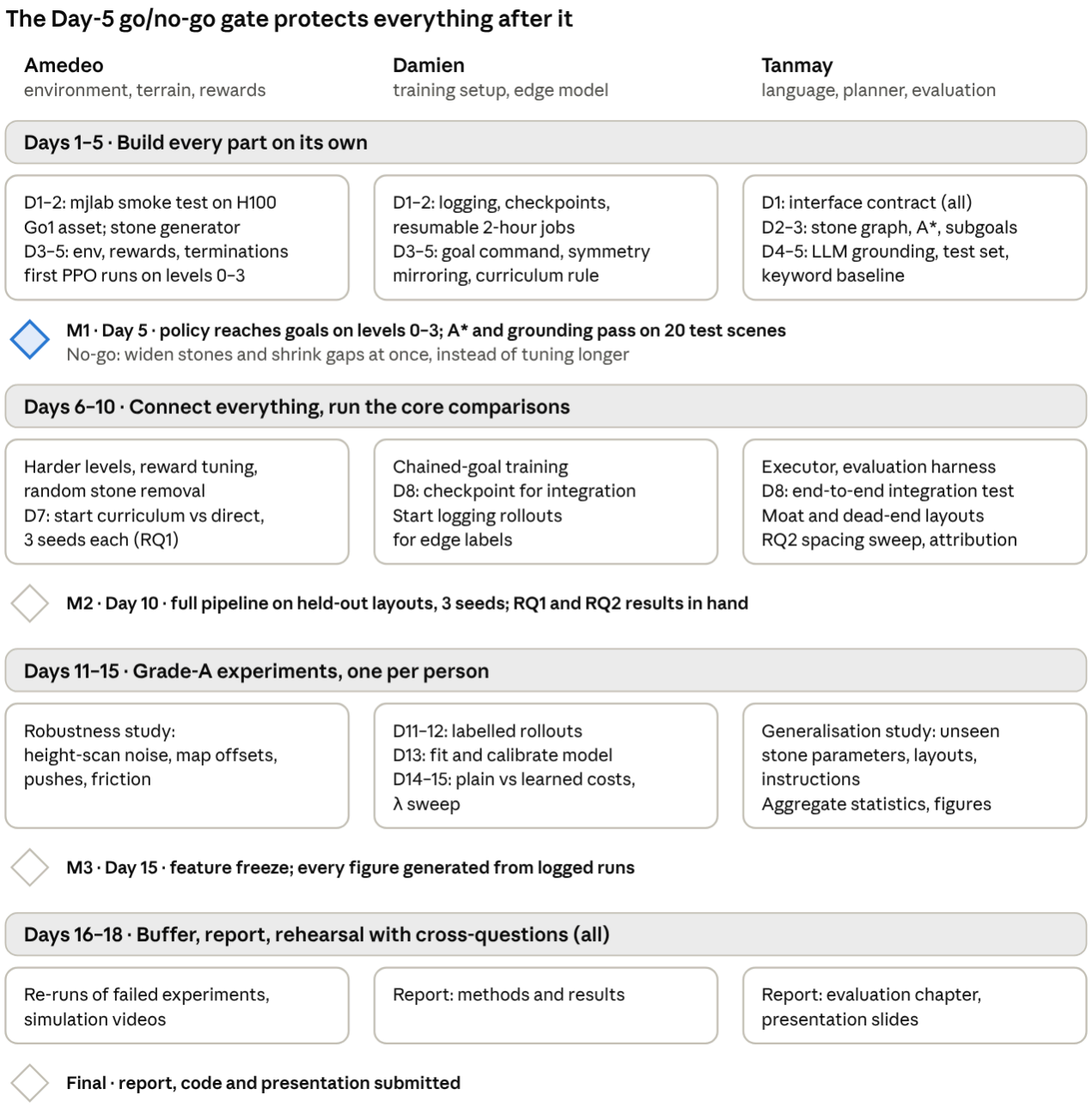
Question	Experiment	Conditions	Owner
RQ1 Controller	curriculum versus direct training on the hardest level, same budget	2 conditions × 3 seeds, tested on open fields at every level	Amedeo, Damien
RQ2 Planning	subgoals versus final goal only, on moat and dead-end fields	final goal only; spacing 0.5, 1 and 2 m	Tanmay
RQ3 System	where failures come from, full system versus oracle language	test layouts and instructions	Tanmay
Grade A	execution-aware edge costs	plain versus learned costs; λ sweep	Damien
Grade A	robustness	height-scan noise, map offsets of 2 to 5 cm, pushes, friction	Amedeo
Grade A	generalisation	seen versus unseen stone parameters, layouts and instructions	Tanmay

"Final goal only" is Zhang et al.'s setting, so RQ2 directly tests when their policy needs a planner. The robustness study checks their finding that a 5 cm odometry drift could make ANYmal fall, on a much smaller robot.

Statistics. Three training seeds per policy condition and at least 200 episodes per seed and test condition. Every condition uses the same layout seeds, so comparisons are paired. Report the mean and standard deviation across training seeds, and Wilson 95% intervals for pooled success rates.

Build order and roadmap

The build runs bottom-up on the critical path and top-down everywhere else. The controller starts on Day 1 with two people, while language and planning are finished early on recorded scene files, so they are ready and tested before the controller is.



roadmap · four phases, three people, four gates

The Day-8 integration test is the first time all layers run together; the chained-goal checkpoint must be ready for it.

Risks and fallbacks

The controller remains the only risk that can sink the project; every other risk has a cheap fallback that keeps the research questions answerable.

Risk	Early warning	Fallback
Controller does not learn sparse footholds in budget	cannot reach goals on levels 0 to 3 by Day 5	widen stones and shrink gaps; drop the hardest levels. RQ2 and RQ3 still run on easier fields
mjlab fails on the H100 partitions	Day-2 smoke test fails, or queues exceed a day	MuJoCo Playground with fewer parallel environments. Language work is unaffected, since it runs on the T4 and results are cached
Chained subgoals are out of distribution	success drops at the Day-8 integration test when subgoals replace single goals	raise the chained-goal share in training, match subgoal spacing to training goal distances, enlarge the reach radius
Grounding is weak on spatial relations	accuracy per category on the test set	move more geometry into precomputed relations, or switch to parse-then-resolve: the model outputs relation and anchor, code picks the platform
Too few failure labels for the edge model	fewer than a few hundred failed edges after Day 12	oversample hard levels, or replace the classifier with a lookup table binned by gap and height difference

Relation to Zhang et al.

The controller replicates their method at student scale; the contribution is the system around it.

Aspect	Zhang et al.	This project
Robot and simulator	ANYmal-D, Isaac Gym, real-robot tests	Go1, mjlab, simulation only
Controller	goal-reaching PPO, curriculum, curiosity, symmetry, generalist then specialists	same formulation, curriculum and symmetry; no curiosity, no specialists; adds chained goals
Terrain	stones everywhere, so the straight line to the goal always works	adds moat and dead-end fields where it does not
Where goals come from	placed by hand	a language instruction, through the scene's object list
Planning	none; waypoints shown as a demo	A* with subgoals, plain and execution-aware costs
Failure analysis	failures noted as hard to interpret	every failure attributed to language, planner or controller

The honest claim for the report: a layered, instruction-following system built on a replicated sparse-terrain controller, and a measurement of when and why that system fails. It is not a new locomotion method.

Sources

- Zhang, Rudin, Hoeller, Hutter, Learning Agile Locomotion on Risky Terrains (arXiv 2311.10484v2)
- mjlab repository and README
- mjlab v1.3.0 release notes (terrain presets, Go1 stair policy)
- mjlab RayCast sensor documentation